

MA 587: Homework 4

Joseph F. Specht IV

April 14, 2026

1 Stability

1.A A-Stability and L-Stability

Before determining the stability of each method, I am going to define the stability criteria. A method being A-stable means the stability function should not have a magnitude greater than 1, which can be expressed as $|\phi(z)| \leq 1$ for the region $\operatorname{Re}(z) < 0$. For a method to be L-stable, it needs to be both A-stable and the stability function $\phi(z)$ to satisfy the following limit:

$$\lim_{z \rightarrow \infty} \phi(z) = 0 \tag{1}$$

We will also let $z = h\lambda$ for each numerical integration method.

Forward Euler

1. We start with the formulation for the forward Euler method.

$$x_{k+1} = x_k + h\lambda x_k = x_k(1 + h\lambda) = x_k\phi(z) \tag{2}$$

$$\phi(z) = 1 + z \tag{3}$$

2. From our notes, we know the stability region is $|1 + z| \leq 1$, which is a disk of radius 1 centered at -1, which does not include $\operatorname{Re}(z) < 0$.
3. Therefore, the forward Euler method is neither A-stable nor L-stable.

Backward Euler

1. We start with the stability function for the backward Euler method.

$$\phi(z) = \frac{1}{1-z} \quad (4)$$

2. The stability function for the backward Euler method is needs to remain stable for all $Re(z) < 0$. We know the A-stability criteria is as follows:

$$\left| \frac{1}{1-z} \right| \leq 1 \quad (5)$$

$$\frac{1}{|1-z|} \leq 1 \quad (6)$$

$$|1-z| \geq 1 \quad (7)$$

3. We see the stability region is $|1-z| \geq 1$, which corresponds to everything outside of a disk of radius 1 centered at 1. We see the backward Euler method is A-stable as the stability region includes $Re(z) < 0$.
4. By taking the following limit, we find the backward Euler method to be L-stable.

$$\lim_{z \rightarrow \infty} \phi(z) = \lim_{z \rightarrow \infty} \frac{1}{1-z} = 0 \quad (8)$$

5. Therefore, the backward Euler method is both A-stable and L-stable.

Crank-Nicolson

1. We start with the formulation for the Crank-Nicolson method.

$$x_{k+1} = x_k + h \left(\frac{\lambda x_k + \lambda x_{k+1}}{2} \right) = x_k + \frac{zx_k + zx_{k+1}}{2} \quad (9)$$

2. Rearrange to obtain the stability function

$$x_{k+1} - \frac{zx_{k+1}}{2} = x_k + \frac{zx_k}{2} \quad (10)$$

$$x_{k+1}(1 - z/2) = x_k(1 + z/2) \quad (11)$$

$$x_{k+1} = x_k \frac{1 + z/2}{1 - z/2} = x_k \phi(z) \quad (12)$$

$$\phi(z) = \frac{1 + z/2}{1 - z/2} \quad (13)$$

3. Next, we find the stability condition as follows,

$$\left| \frac{1 + z/2}{1 - z/2} \right| \leq 1 \quad (14)$$

$$|1 + z/2| \leq |1 - z/2| \quad (15)$$

4. We see the stability condition holds $\forall \text{Re}(z) < 0$, so we conclude the Crank-Nicolson method is A-stable.

5. By taking the following limit, we find the limit of the stability function of the Crank-Nicolson method equals -1 and not 0. Thus, we conclude the Crank-Nicolson method is not L-stable.

$$\lim_{z \rightarrow \infty} \phi(z) = \lim_{z \rightarrow \infty} \frac{1 + z/2}{1 - z/2} = \frac{z/2}{-z/2} = -1 \quad (16)$$

6. Therefore, the Crank-Nicolson method is A-stable, but not L-stable.

1.B Stiff ODE

Backward Euler Formulation

1. We begin with the standard backward Euler equation.

$$x_{k+1} = x_k + h\lambda x_{k+1} \quad (17)$$

2. We know $\dot{x} = \lambda x$, which we use to rewrite the backward Euler equation as

$$x_{k+1} = x_k + h \{ C[x_{k+1} - \sin(t_{k+1})] + \cos(t_{k+1}) \} \quad (18)$$

3. Rearrange to isolate x_{k+1}

$$x_{k+1} = x_k + h\dot{x}_{k+1} = x_k + hCx_{k+1} - hC \sin(t_{k+1}) + h \cos(t_{k+1}) \quad (19)$$

$$x_{k+1} - hCx_{k+1} = x_k - hC \sin(t_{k+1}) + h \cos(t_{k+1}) \quad (20)$$

$$x_{k+1}(1 - hC) = x_k - hC \sin(t_{k+1}) + h \cos(t_{k+1}) \quad (21)$$

$$x_{k+1} = \frac{x_k - hC \sin(t_{k+1}) + h \cos(t_{k+1})}{1 - hC} \quad (22)$$

Crank-Nicolson Formulation

1. We begin with the standard Crank-Nicholson equation.

$$x_{k+1} = x_k + h\lambda \left(\frac{x_k + x_{k+1}}{2} \right) \quad (23)$$

$$x_{k+1} = x_k + \frac{h}{2} (\lambda x_k + \lambda x_{k+1}) \quad (24)$$

2. Once again, use the fact that $\dot{x} = \lambda x$.

$$x_{k+1} = x_k + \frac{h}{2} \{C [x_k - \sin(t_k)] + \cos(t_k) + C [x_{k+1} - \sin(t_{k+1})] + \cos(t_{k+1})\} \quad (25)$$

3. Rearrange to isolate x_{k+1} .

$$x_{k+1} - \frac{hC}{2} x_{k+1} = x_k + \frac{h}{2} \{C [x_k - \sin(t_k)] + \cos(t_k) - C \sin(t_{k+1}) + \cos(t_{k+1})\} \quad (26)$$

$$x_{k+1} \left(1 - \frac{hC}{2} \right) = x_k + \frac{h}{2} \{C [x_k - \sin(t_k)] + \cos(t_k) - C \sin(t_{k+1}) + \cos(t_{k+1})\} \quad (27)$$

$$x_{k+1} = \frac{x_k + \frac{h}{2} \{C [x_k - \sin(t_k)] + \cos(t_k) - C \sin(t_{k+1}) + \cos(t_{k+1})\}}{1 - \frac{hC}{2}} \quad (28)$$

$$x_{k+1} = \frac{(1 + \frac{Ch}{2})x_k - \frac{Ch}{2} [\sin(t_k) + \sin(t_{k+1})] + \frac{h}{2} [\cos(t_k) + \cos(t_{k+1})]}{1 - \frac{hC}{2}} \quad (29)$$

Results After coding the previous methods in python, we obtain the following errors.

Table 1: Relative Error in Numerical Integration Techniques (Approximation - Exact)

T [s]	Backward Euler Error	Crank-Nicolson Error
0	0	0
0.1	-3.02773e-04	-7.56554e-05
0.2	-9.69622e-04	-1.38141e-04
0.3	-1.91389e-03	-1.90123e-04
0.4	-3.07432e-03	-2.33601e-04
0.5	-4.40743e-03	-2.70097e-04

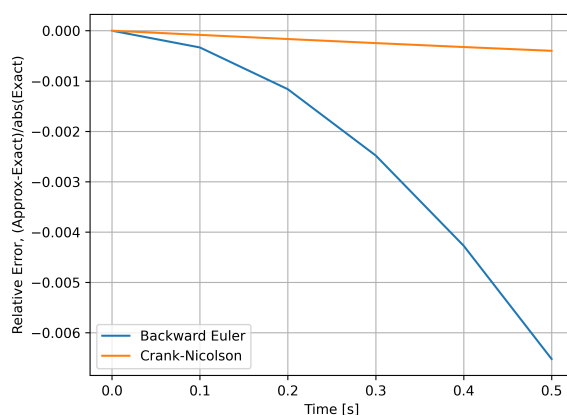


Figure 1: Error in Numerical Integration Techniques (Approximation - Exact)

From our results, we see the Crank-Nicolson method better approximates the exact solution than the Backward Euler method. However, both methods underestimate the true solution. Speaking from a nuclear engineering perspective, we would prefer methods that *overestimate* the true solution as they provide a more conservative estimate for safety calculations.

2 1D Heat Equation

2.A Standard Continuous Weak Form

1. To find the standard continuous weak form, first multiply both sides of the heat equation by some test function $v \in V_h$.

$$(u_t - u_{xx})v = fv \quad (30)$$

2. Then, integrate both sides

$$\int_0^1 dx(u_t - u_{xx})v = \int_0^1 dx fv \quad (31)$$

$$\int_0^1 dx u_t v - \int_0^1 dx u_{xx} v = \int_0^1 dx fv \quad (32)$$

3. Via integration by parts,

$$-\int_0^1 dx u_{xx} v dx = \int_0^1 dx u_x v_x - u_x v|_0^1 \quad (33)$$

4. Where the second term equals zero because of the boundary conditions imposed by the Sobolev space.

$$-u_x v|_0^1 = 0 \quad (34)$$

5. Substituting our new term back into the original expression

$$\int_0^1 dx u_t v + \int_0^1 dx u_x v_x dx = \int_0^1 dx fv \quad (35)$$

6. We then write the previous equation in the standard continuous weak formulation

$$(u_t, v) + a(u, v) = L(v) \quad (36)$$

where

- (a) $(u_t, v) = \int_0^1 dx u_t v$ is the bilinear mass term
- (b) $a(u, v) = \int_0^1 dx u_x v_x dx$ is the bilinear stiffness form
- (c) $L(v) = \int_0^1 dx fv$ is the linear form or the load term

2.B L2-Projection

1. If the approximation is optimal, then we know $\hat{u}_0 \in V_h$ because $\|v - u_0\|_{L2}^2$ is a norm and is, consequently, a non-negative, real number.
2. The optimal approximation would be when the following expression is equal to zero or when the solution $u_0 \in V_h$.

$$\min_{v \in V_h} \|v - u_0\|_{L2}^2 = 0 \quad (37)$$

3. Because $\hat{u}_0 \in V_h$, we can write.

$$\hat{u}_0 = \sum_{j=1}^M c_j \phi_j \quad (38)$$

4. We are trying to minimize the square of the L2 error of the following expression.

$$\|\hat{u} - u_0\|_{L2}^2 = \int_0^1 dx \left(\sum_{j=1}^M c_j \phi_j - u_0 \right)^2 \quad (39)$$

5. To minimize, take the derivative with respect to each coefficient and set the expression equal to zero.

$$\frac{\partial}{\partial c_i} (\|\hat{u} - u_0\|_{L2}^2) = 2 \int_0^1 dx \left(\sum_{j=1}^M c_j \phi_j - u_0 \right) \phi_i = 0 \quad (40)$$

6. Rearrange

$$\int_0^1 dx \left(\sum_{j=1}^M c_j \phi_j \right) \phi_i = \int_0^1 dx u_0 \phi_i \quad (41)$$

$$\sum_{j=1}^M c_j (\phi_j, \phi_i) = (u_0, \phi_i) \quad (42)$$

7. We need to isolate c_j to find an expression for $\hat{u}_0$. Convert the previous expression to matrix notation

$$\mathbf{M}\mathbf{c} = \mathbf{b} \quad (43)$$

where,

(a) $\mathbf{M}$ is an $M \times M$, tridiagonal matrix mass matrix where $M_{ij} = \int_0^1 dx \phi_j \phi_i$

(b) $\mathbf{c}$ is an $M \times 1$ coefficient vector

(c) $\mathbf{b}$ is an $M \times 1$ load vector where $b_i = \int_0^1 dx u_0 \phi_i$

8. Inverting the matrix gives our expression for $\mathbf{c}$.

$$\mathbf{c} = \mathbf{M}^{-1}\mathbf{b} \quad (44)$$

9. Using this expression, we can write $\hat{u}_0$ as

$$\hat{u}_0 = \sum_{j=1}^M (\mathbf{M}^{-1}\mathbf{b})_j \phi_j \quad (45)$$

once again, where

(a) $\mathbf{M}$ is an $M \times M$, tridiagonal matrix where $M_{ij} = \int_0^1 dx \phi_j \phi_i$

(b) $\mathbf{b}$ is an $M \times 1$ vector where $b_i = \int_0^1 dx u_0 \phi_i$

2.C Semi-Discrete Galerkin Approximation

1. The semi-discrete Galerkin approximation is used for PDEs which are functions of space and time. The approximation discretizes the PDE in space, but leaves time continuous.

2. According to the problem statement, we are trying to find a function u_h that satisfies the following,

$$(u_{t,h}, v_h) + a(u_h, v_h) = (f, v_h) \quad (46)$$

3. Using the SDG approximation, we can write u_h as the sum of the product of a weight that evolves in time, $T_j(t)$, and a shape function that evolves in space, $\phi_j(x)$ (the same ϕ basis vectors from previous parts).

$$u_h(x, t) = \sum_{j=1}^M U_j(t) \phi_j(x) \quad (47)$$

4. Next, plug the previous approximation into the weak formulation we found in part a. In honor of Euler and working with the forward and backward Euler methods in this homework, I will use Euler notation for the partial derivatives with respect to time and space.

$$\int_0^1 dx \frac{\partial}{\partial t} \left[\sum_{j=1}^M U_j(t) \phi_j(x) \right] v_h + \int_0^1 dx \frac{\partial}{\partial x} \left[\sum_{j=1}^M U_j(t) \phi_j(x) \right] v_{h,x} = \int_0^1 dx f v_h \quad (48)$$

5. As we know $v_h \in V_h = \text{span}\{\phi_1, \dots, \phi_M\}$, we know v_h can always be represented as the linear combination of some ϕ vectors. Due to linear nature of the problem, if we find a random ϕ_i to be a solution

to the problem, we can always represent v_h as the linear combination of such basis vectors. Therefore, solving the following equation would allow us to find such a u_h we are looking for.

$$\int_0^1 dx \frac{\partial}{\partial t} \left[\sum_{j=1}^M U_j(t) \phi_j(x) \right] \phi_i(x) + \int_0^1 dx \frac{\partial}{\partial x} \left[\sum_{j=1}^M U_j(t) \phi_j(x) \right] \phi_{x,i}(x) dx = \int_0^1 dx f \phi_i(x) \quad (49)$$

6. Rearranging some terms,

$$\sum_{j=1}^M U_{t,j}(t) \int_0^1 dx \phi_j(x) \phi_i(x) + \sum_{j=1}^M U_j(t) \int_0^1 dx \phi_{x,j}(x) \phi_{x,i}(x) = \int_0^1 dx f \phi_i(x) \quad (50)$$

7. We can rewrite each term using the previous definition for M_{ij} , the definition of the linear form, and the definition of the bilinear form.

$$\sum_{j=1}^M U_{t,j}(t) M_{ij} + \sum_{j=1}^M U_j(t) a(\phi_i, \phi_j) = L(\phi_i) \quad (51)$$

8. We can then express the previous equation in matrix notation

$$\mathbf{M}\mathbf{U}_t + \mathbf{A}\mathbf{U} = \mathbf{F} \quad (52)$$

where,

- (a) $\mathbf{M}$ is an $M \times M$ tridiagonal matrix mass matrix where $M_{ij} = \int_0^1 dx \phi_j \phi_i$,
- (b) $\mathbf{U}_t$ is an $M \times 1$ column vector representing the time rate of change of the temperature distribution,
- (c) $\mathbf{A}$ is an $M \times M$ stiffness matrix where $A_{ij} = a(\phi_i, \phi_j)$,
- (d) $\mathbf{U}$ is an $M \times 1$ column vector representing the current temperature distribution,
- (e) $\mathbf{F}$ is an $M \times 1$ load vector

9. I should note, the order of the matrix multiplication is as such because $\mathbf{M}\mathbf{U}_t$ and $\mathbf{A}\mathbf{U}$ are $M \times 1$ column vectors, which have the same dimensions as the load vector. Conversely, $\mathbf{U}_t\mathbf{M}$ and $\mathbf{U}\mathbf{A}$ are not defined.

10. We can then rearrange the equation to solve for $\mathbf{U}_t$

$$\mathbf{U}_t = \mathbf{M}^{-1}(\mathbf{F} - \mathbf{A}\mathbf{U}) \quad (53)$$

11. To determine if the above equation is solvable, we need to prove $\mathbf{M}$ is invertible. I will now show $\mathbf{M}$ is a symmetric positive definite matrix, which are always invertible.

(a) First, we can show $\mathbf{M}$ is symmetric.

$$M_{ij} = \int_0^1 dx \phi_j(x) \phi_i(x) = \int_0^1 dx \phi_j(x) \phi_i(x) = M_{ji} \quad (54)$$

(b) Next, we need to show $\mathbf{x}^T \mathbf{M} \mathbf{x} > 0$ for any non-zero $\mathbf{x} \in \mathbb{R}^M$. We start by breaking the matrix equation down into each component.

$$\mathbf{x}^T \mathbf{M} \mathbf{x} = \sum_{i=1}^M \sum_{j=1}^M x_i \left[\int_0^1 dx \phi_j \phi_i \right] x_j \quad (55)$$

(c) Rearranging and rewriting as an inner product,

$$= \int_0^1 dx \sum_{i=1}^M x_i \phi_i \sum_{j=1}^M \phi_j x_j = \left(\sum_{i=1}^M x_i \phi_i, \sum_{j=1}^M x_j \phi_j \right) \quad (56)$$

(d) We see $x_i \phi_i \in V_h$, which is just some function in the vector space of our problem. Additionally, we see both terms are the same aside from the index notation. Because both terms are the same, we can write the following,

$$\mathbf{x}^T \mathbf{M} \mathbf{x} = \left(\sum_{i=1}^M x_i \phi_i, \sum_{j=1}^M x_j \phi_j \right) = \left\| \sum_{i=1}^M x_i \phi_i \right\|_{L2} \quad (57)$$

(e) Because $\mathbf{x}^T \mathbf{M} \mathbf{x}$ is equal to the L2 norm of some vector $\in V_h$, we know it is non-negative. We also know this expression cannot be equal to zero because $\mathbf{x} \neq \vec{0}$ and $\phi_i \neq 0$. We see $\mathbf{M}$ is a positive definite matrix.

(f) We have shown $\mathbf{M}$ is SPD, which means $\mathbf{M}$ will always be invertible.

12. As $\mathbf{M}$ is SPD and, thus, invertible, we find our previous equation to be consistent (in the linear algebra sense). Therefore, we know at least one solution exists.

$$\mathbf{U}_t = \mathbf{M}^{-1}(\mathbf{F} - \mathbf{A}\mathbf{U}) \quad (58)$$

13. For u_h to satisfy the weak form from part a, we needed to show the first derivatives with respect to time and space existed and were obtainable. The first derivative wrt space is obviously obtainable via the definition of our hat basis functions. By showing the above equation is consistent, we showed the first derivative wrt time is also obtainable.

14. We can then express the first derivative wrt time, $u_{t,h}$, as follows,

$$u_{t,h} = \sum_{i=1}^M U_{t,i}(t) \phi_i(x) \quad (59)$$

where,

(a) $U_{t,i}$ is the i 'th element of $\mathbf{U}_t$

15. We have shown $u_h(t) \in V_h$ satisfies the weak form for any basis vector in V_h , ϕ_i . Due to the linearity of the weak form, we conclude $u_h(t) \in V_h$ also satisfies the weak form for all test functions $v_h \in V_h$.

3 Reduced Order Model

3.A Describing Projection-Based ROM using the POD basis

If we have a problem where the dimension M is very large, we will run into a computational bottleneck when trying to calculate the Jacobian. To solve such problems, we need to calculate then **invert** the Jacobian at each time step. We know $J \in \mathbb{R}^{M \times M}$ and, if M is exceedingly large, inverting the Jacobian once, let alone at each time step, is computationally intractable. Forgoing numerical techniques that do not require the calculation and inversion of a Jacobian (e.g. the Jacobian-Free Newton-Krylov method), we can find some way to **reduce the dimensionality** of our problem.

We might posit there is such a method where we can reduce the dimensionality of our problem such that we only have to calculate an $m \times m$ Jacobian and not an $M \times M$ Jacobian, where $m \ll M$. If we suppose our problem has extraneous degrees of freedom (DOFs), we can intuit there are m degrees of freedom required to represent the bulk of our solution (say 99% for example, but this percent is at the discretion of the practitioner). We also posit there is some function capable of **projecting** our M -dimension problem into a new, **reduced-order**, m -dimension problem. We can call this projection an **encoder** such that the encoder operating on our problem maps $\mathbb{R}^M \rightarrow \mathbb{R}^m$.

Intuitively, we expect the physics of the problem not to change. If we perform the same physics on the reduced order model our encoder generated, we expect the reduced-order solution to make up the most important parts of the full-order solution. Therefore, we can perform the same **dynamics** on our reduced-order problem to obtain a reduced-order solution. When finding the lowest m such that we can represent our problem sufficiently, it makes sense for us to use the fewest DOFs possible.

Suppose our solution $\in \mathbb{R}^m$ is the lowest order solution possible. The solution would not be the lowest order possible if any component of our solution could be represented as the linear combination of other components of our solution. Therefore, we can say the lowest order solution is comprised of m **orthogonal vectors** that span $\mathbb{R}^m$. Additionally, we can solidify our use of the word "projection" earlier. We assumed our encoder removes the extraneous degrees of freedom ($M - m$) from a problem and returns the lowest-order (m) model capable of properly representing our problem. If we apply the encoder to our reduced-order problem, there are no extraneous degrees of freedom to remove, so we are left with the same, m -dimension problem. Thus, the encoder is **idempotent** and is indeed a **proper projection**.

Since we are actually looking for a solution in $\mathbb{R}^M$, we need to map our reduced-order solution back into $\mathbb{R}^M$. We call the map that takes the reduced-order solution from $\mathbb{R}^m \rightarrow \mathbb{R}^M$ the **decoder**. We argue the decoder is a **proper projection** like the encoder as applying the decoder to a full-order solution will not change the dimension of the solution (i.e. the decoder is **idempotent**).

3.B Reduced-Order Galerkin Matrices

1. From the previous part, we know the reduced-order Galerkin matrices are $\in \mathbb{R}^{M \times m}$.
2. We simply write the reduced-order matrices as such.

$$\tilde{M} = \Phi_K^T M_h \Phi_K \quad (60)$$

$$\tilde{K} = \Phi_K^T K_h \Phi_K \quad (61)$$

3.C FEniCSx Code

Chat-GPT Prompt I am solving a finite element math homework. Attached is the question prompt [paste screenshot of the question]. Write code that solves part c using the FEniCSx framework. Additionally, make sure to include the following changes:

- Use ‘dolfinx.fem.petsc.assemble__matrix’ and NOT ‘fem.petsc.assemble__matrix’. This is a compatibility issue with FEniCSx and PETSc.
- Ensure there are no compatibility issues when mixing NumPy and UFL expressions. Ensure all spatial expressions are evaluated consistently at DOF level.
- Do not use the invalid ‘getDenseArray()’ on sparse PETSc matrices. Instead, use ‘A.convert("dense").getDenseArray()’ to ensure compatibility with PETSc AIJ matrix format.
- Do not use ‘scipy’ as a dependency. My conda environment cannot resolve the installation of scipy.
- For the line "the total should be 10^4 cases," this does not need solve 100,000 cases, but that the mesh grid should have 100,000 elements.
- Precompute the reduced operators for a more efficient solve.
- When comparing the models, select 9 random parameter pairs and compute the relative L2 error between the FOM and ROM.
- Add structured runtime comparison that calculates the average solve time for the FOM and ROM, the total ROM evaluation time, and calculate the speedup factor obtained by the ROM.
- Print all the errors as a LaTeX table so it can be directly copy pasted into LaTeX.
- Ensure a consistent use of Crank-Nicholson discretization in both the FOM and ROM.
- Organize the script into logical sections that are easily understood by an outside observer.

Results Comparing the average L2 error between the two models yields an average error of 2.6469e-3. The ROM shows remarkably close agreement with the FOM and the dimensionality reduction from $M = 1000 \rightarrow m = 15$ did not diminish the solution quality substantially. The speedup obtained by using the ROM was a factor of 17.72. You can always decrease the number of dimensions of the ROM, but this comes at the cost of accuracy. Theoretically, there will be a minimum ROM dimension where the relative error between the two models will be above some user-defined error.

Table 2: 9 Randomly Sampled Points and the Associated L2 Error

μ	σ	Relative L^2 Error
0.6263	0.5202	2.973563e-03
0.4646	0.7788	3.688045e-03
0.1717	0.3505	3.387674e-03
0.4747	0.4394	2.340601e-03
0.4545	0.2697	6.524803e-04
0.6364	0.4232	2.536409e-03
0.0505	0.7141	3.654303e-03
0.5253	0.1162	1.287322e-03
0.6364	0.6091	3.302224e-03